DevOps Lab Assessment - Experiment 8

Continuous Deployment to Kubernetes via CI/CD Pipeline Automation

STUDENT NAME:	**Rohit**	**ROLL NUMBER:**	**2301730179**
EMAIL ADDRESS:	rohit2301730179@gmail.com	**ASSESSMENT:**	Question 8 (5 Marks)
COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Submitted by **Rohit** (Roll No: **2301730179**, Email: **rohit2301730179@gmail.com**). Continuous Deployment (CD) automates the transition of validated code from source control directly to production Kubernetes clusters. The objective of this experiment is to architect and execute an automated CI/CD pipeline using **GitHub Actions** that compiles, tests, and packages a containerized application, publishes versioned images to an OCI registry, dynamically updates Kubernetes manifests via **Kustomize**, applies manifests to a cluster using kubectl, and automates rollback if health checks fail.

2. Production CI/CD Pipeline Manifest (.github/workflows/deploy.yml)

```
// File: .github/workflows/deploy.yml | Candidate: Rohit (2301730179)

# Author: Rohit (Roll: 2301730179 | rohit2301730179@gmail.com) name: CI/CD Pipeline - K8s Automated CD on: push: branches: [main] jobs: build-and-push: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: docker/login-action@v3 with: { username: ${{ secrets.DOCKER_USER }}, password: ${{ secrets.DOCKER_TOKEN }} } - name: Build and Push Docker Image run: | IMAGE_TAG=rohit2301730179/devops-microservice:${{ github.sha }} docker build -t $IMAGE_TAG -f src/app/Dockerfile src/app docker push $IMAGE_TAG deploy-to-k8s: needs: build-and-push runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: azure/k8s-set-context@v4 with: { method: kubeconfig, kubeconfig: ${{ secrets.KUBECONFIG_DATA }} } - name: Dynamically Update Manifests & Deploy run: | cd src/k8s kustomize edit set image devops-microservice=rohit2301730179/devops-microservice:${{ github.sha }} kubectl apply -k . kubectl rollout status deployment/web-app-deployment --timeout=90s - name: Automated Rollback on Failure if: failure() run: kubectl rollout undo deployment/web-app-deployment
```

3. Step-by-Step CI/CD Execution & Deployment Logs

```
$ git commit -m 'feat: candidate Rohit release v1.0.1' && git push origin main
[main a48f102] feat: candidate Rohit release v1.0.1 2 files changed, 14 insertions(+), 2 deletions(-) To github.com:rohit-2301730179/devops-lab-2.git --> GitHub Actions Workflow Triggered: Run #42 (Commit: a48f102)
```

```
$ kubectl apply -k src/k8s/ kubectl rollout status deployment/web-app-deployment
deployment.apps/web-app-deployment configured Waiting for deployment rollout to finish: 1 out of 3 new replicas updated... Waiting for deployment rollout to finish: 2 out of 3 new replicas updated... deployment "web-app-deployment" successfully rolled out
```

4. Engineering Observations & Conclusion

Key Technical Takeaways

1. **Candidate:** Pipeline engineered and demonstrated by **Rohit** (Roll No: **2301730179**).
2. **Immutability via Git SHA:** Using ${{ github.sha }} tags eliminates caching ambiguities, guaranteeing that the exact tested commit is deployed.
3. **Declarative GitOps Delivery:** Kustomize modified the manifest programmatically without modifying source templates.
4. **Automated Self-Recovery:** The if: failure() trigger ensures that any deployment stalling past 90 seconds triggers kubectl rollout undo, restoring cluster stability automatically.